جامعة حفر الباطن
University of Hafr Al Batin

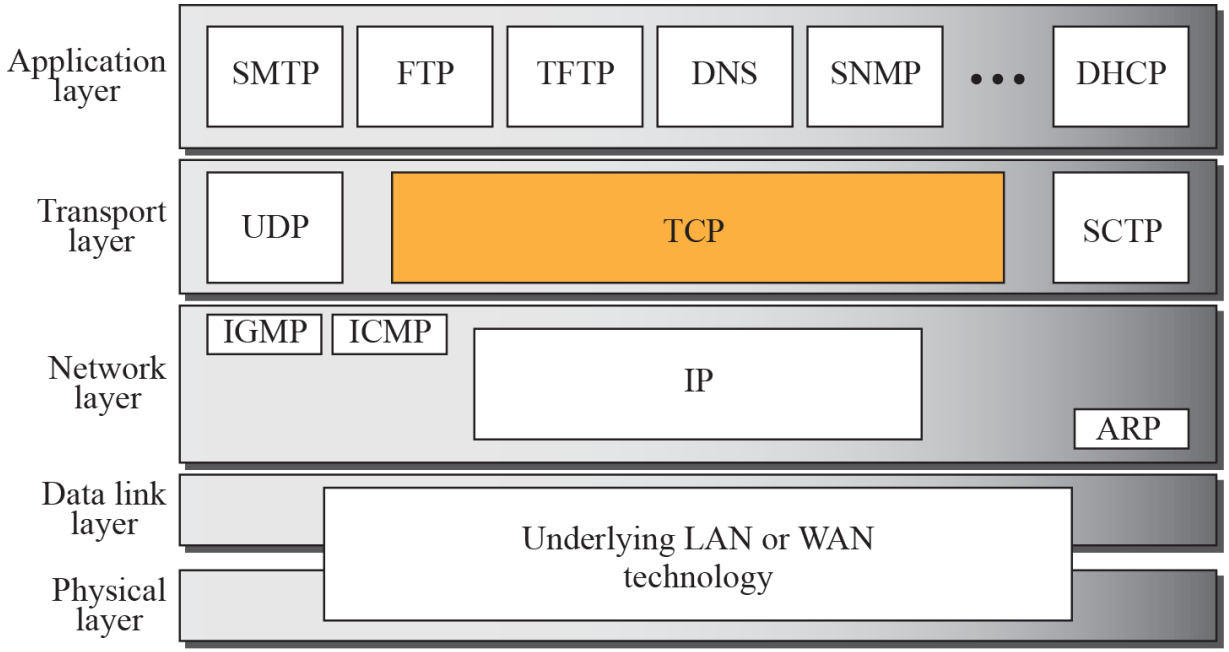
Lecture 10 - Transmission Control Protocol (TCP)

Objectives

- TCP Services.
- TCP Buffers.
- TCP Segment and Format.
- TCP Features
- TCP Connection.
- TCP Flow Control.
- TCP Error Control.
- TCP Congestion Control.

TCP

- TCP lies between the application layer and the network layer, and serves as the intermediary between the application programs and the network operations.



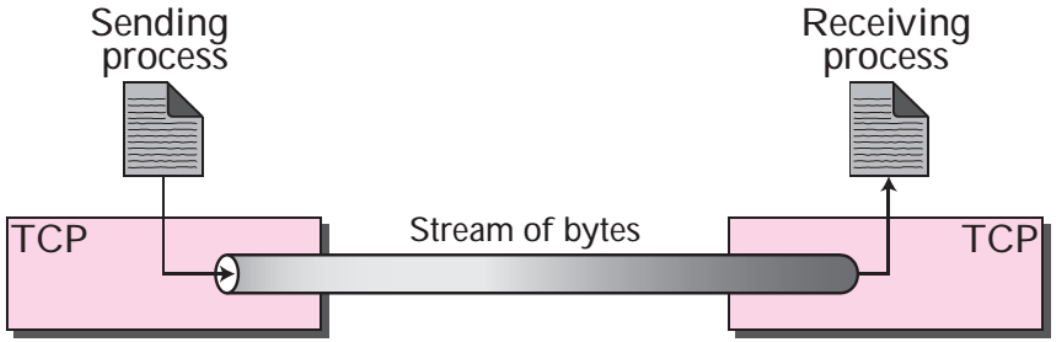
TCP Services

- TCP offers some services to the processes at the application layer.
- As with UDP, TCP provides process-to-process communication using port numbers.
- This table lists some well-known port numbers used by TCP.

Port	*Protocol*	*Description*
7	Echo	Echoes a received datagram back to the sender
9	Discard	Discards any datagram that is received
11	Users	Active users
13	Daytime	Returns the date and the time
17	Quote	Returns a quote of the day
19	Chargen	Returns a string of characters
20 and 21	FTP	File Transfer Protocol (Data and Control)
23	TELNET	Terminal Network
25	SMTP	Simple Mail Transfer Protocol
53	DNS	Domain Name Server
67	BOOTP	Bootstrap Protocol
79	Finger	Finger
80	HTTP	Hypertext Transfer Protocol

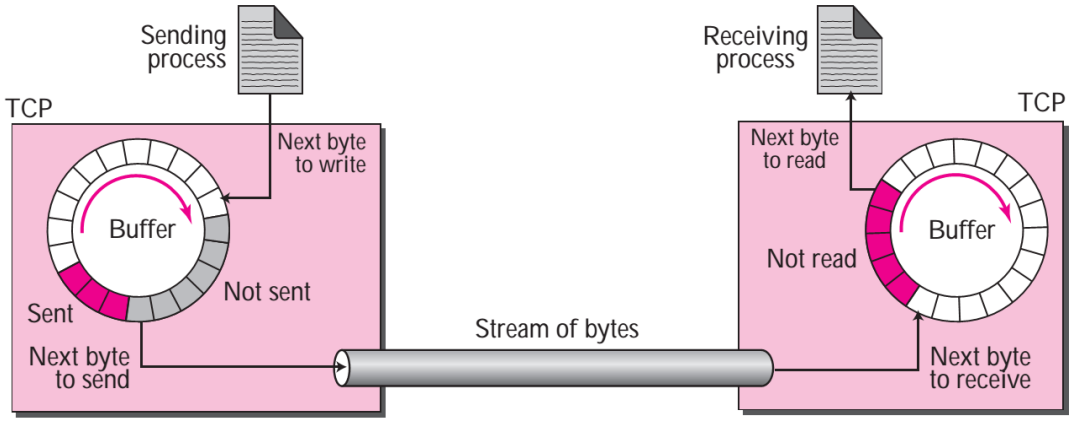
TCP Stream Delivery

- TCP, unlike UDP, allows the sending process to deliver data as a stream of bytes and allows the receiving process to obtain data as a stream of bytes.
  - **TCP, unlike UDP, is a stream-oriented protocol.**
- TCP creates an environment in which the two processes seem to be connected by an *imaginary "tube"* that carries their bytes across the Internet.



TCP Buffers

- Because the sending and the receiving processes may not necessarily write or read data at the same rate, *TCP needs buffers for storage*.
- There are two buffers, the **sending buffer** and the **receiving buffer**, one for each direction.
- One way to implement a buffer is to use a circular array of 1-byte locations.

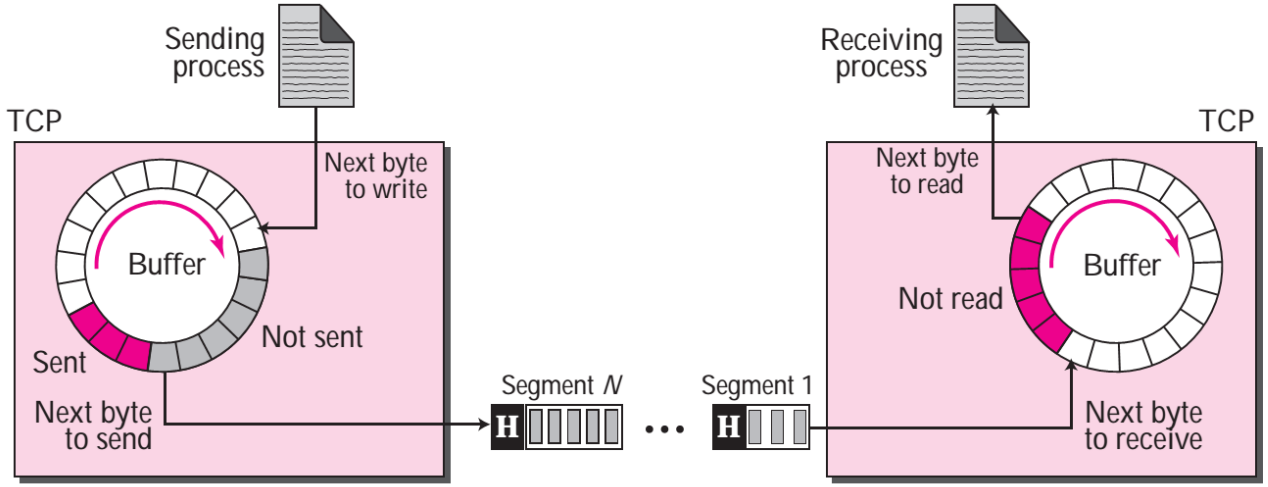


TCP Segments

- Although buffering handles the disparity between the speed of the producing and consuming processes, we need one more step before we can send data.
- The IP layer (as a service provider for TCP) **needs to send data in packets, not as a stream of bytes**.
- To solve this, we do the following:
  1. TCP groups a number of bytes together into a packet called a segment.
  2. TCP adds a header to each segment (for control purposes).
  3. TCP delivers the segment to the IP layer for transmission.
  4. The segments are encapsulated in an IP datagram and transmitted.
- This entire operation is *transparent* to the receiving process.

TCP Segments

- This figure shows how segments are created from the bytes in the buffers.
- Note that *segments are not necessarily all the same size*.



TCP Features

1. Numbering System:

- Although the TCP software keeps track of the segments being transmitted or received, there is no field for a segment number value in the segment header.
- Instead, there are two fields called the sequence number and the acknowledgment number.
- These two fields refer to a **byte number** and not a segment number.

TCP Features

1. Numbering System:

- TCP numbers all data bytes (octets) that are transmitted in a connection.
  - The numbering starts with an arbitrarily generated number.
- **Sequence Number:** is the number of the first byte of data carried in that segment.
- **Acknowledgment Number:** is the number of the next byte that the party expects to receive.

TCP Features

Question:

- Suppose a TCP connection is transferring a file of 5,000 bytes. The first byte is numbered 10,001. What are the sequence numbers for each segment if data are sent in five segments, each carrying 1,000 bytes?

Answer:

Segment 1	→	**Sequence Number:**	**10,001**	**Range:**	**10,001**	to	**11,000**
Segment 2	→	**Sequence Number:**	**11,001**	**Range:**	**11,001**	to	**12,000**
Segment 3	→	**Sequence Number:**	**12,001**	**Range:**	**12,001**	to	**13,000**
Segment 4	→	**Sequence Number:**	**13,001**	**Range:**	**13,001**	to	**14,000**
Segment 5	→	**Sequence Number:**	**14,001**	**Range:**	**14,001**	to	**15,000**

TCP Features

2. Flow Control:

- TCP, unlike UDP, provides flow control.
  - The sending TCP controls how much data can be accepted from the sending process.
  - The receiving TCP controls how much data can be sent by the sending TCP.
- This is done to prevent the receiver from being overwhelmed with data.

TCP Features

3. Error Control:

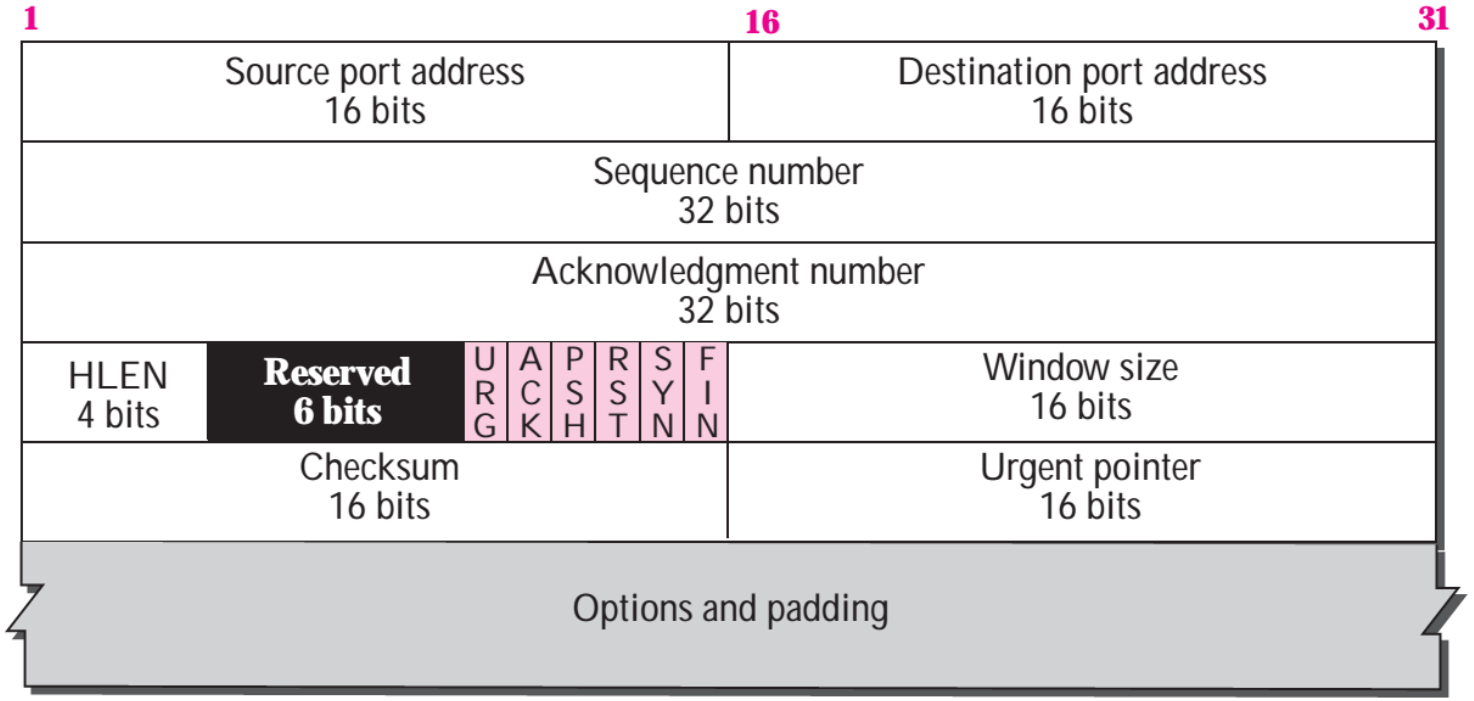
- To provide reliable service, TCP implements an error control mechanism.
- Although error control considers a segment as the unit of data for error detection (loss or corrupted segments), error control is byte-oriented.

4. Congestion Control:

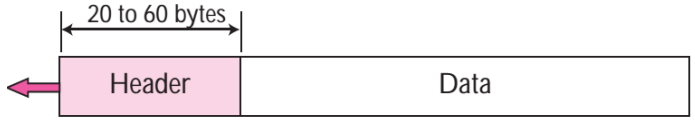
- TCP, unlike UDP, takes into account congestion in the network.
  - The amount of data sent by a sender is not only controlled by the receiver (flow control), but is also determined by the level of congestion, if any, in the network.

TCP Segments

- A packet in TCP is called a **segment**.



b. Header

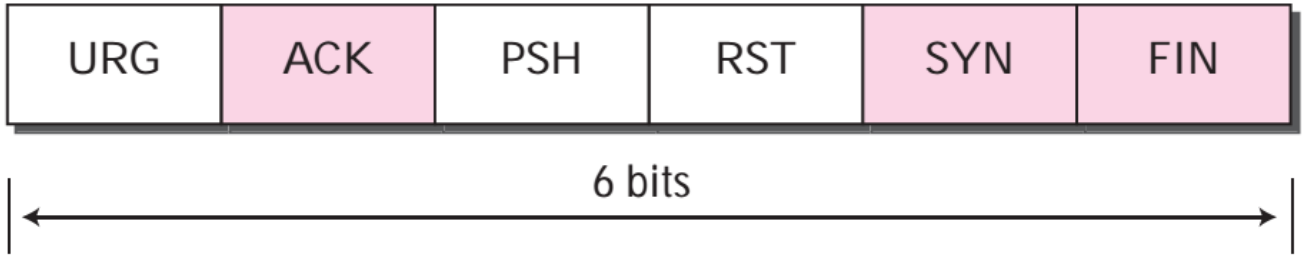


a. Segment

TCP Segments

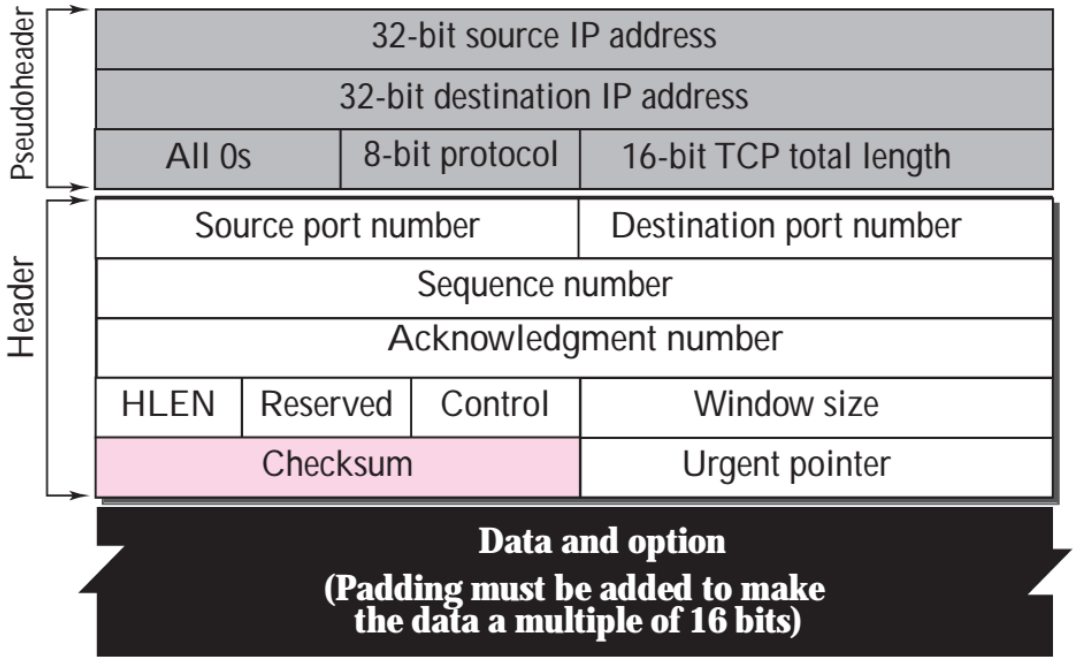
- Control Flags (Bits):

URG: Urgent pointer is valid
ACK: Acknowledgment is valid
PSH: Request for push
RST: Reset the connection
SYN: Synchronize sequence numbers
FIN: Terminate the connection



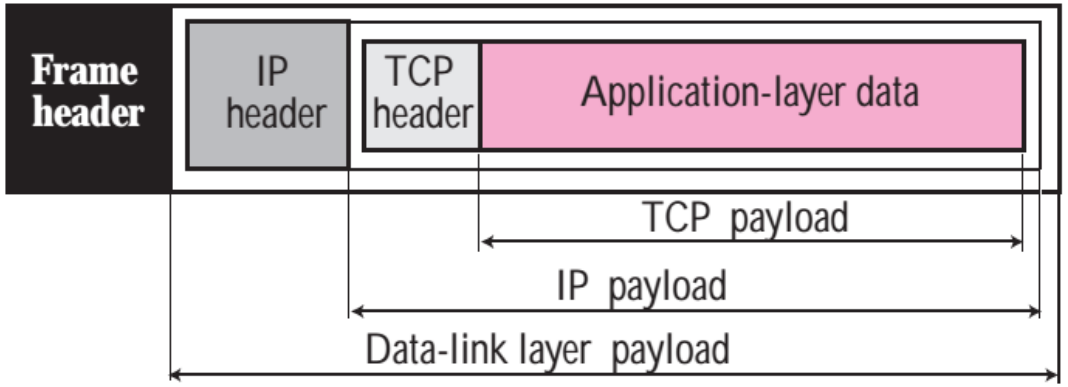
TCP Segments: Checksum

- The calculation of the checksum for TCP follows the same procedure as the one described for UDP.
- However, the use of the checksum in the UDP datagram is optional.
- Whereas the use of the **checksum for TCP is mandatory.**



TCP Encapsulation

- A TCP segment encapsulates the data received from the application layer.
- The TCP segment is encapsulated in an IP datagram.
- Which in turn is encapsulated in a frame at the data-link layer.



TCP Connection

- TCP is connection-oriented .
  - TCP establishes a virtual path between the source and destination.
  - All of the segments belonging to a message are then sent over this virtual path.
  - This facilitates the acknowledgment process as well as retransmission of damaged or lost frames.
- **You may wonder how TCP, which uses the services of IP, a connectionless protocol, can be connection-oriented?**
- The point is that a TCP connection is virtual, not physical.
- TCP operates at a higher level.
- TCP uses the services of IP to deliver individual segments to the receiver, but it controls the connection itself.

TCP Connection

- If a segment is lost or corrupted, it is retransmitted. IP is unaware of this retransmission.
- If a segment arrives out of order, TCP holds it until the missing segments arrive; IP is unaware of this reordering.
- In TCP, connection-oriented transmission requires three phases:
  1. Connection establishment.
  2. Data transfer.
  3. Connection termination.

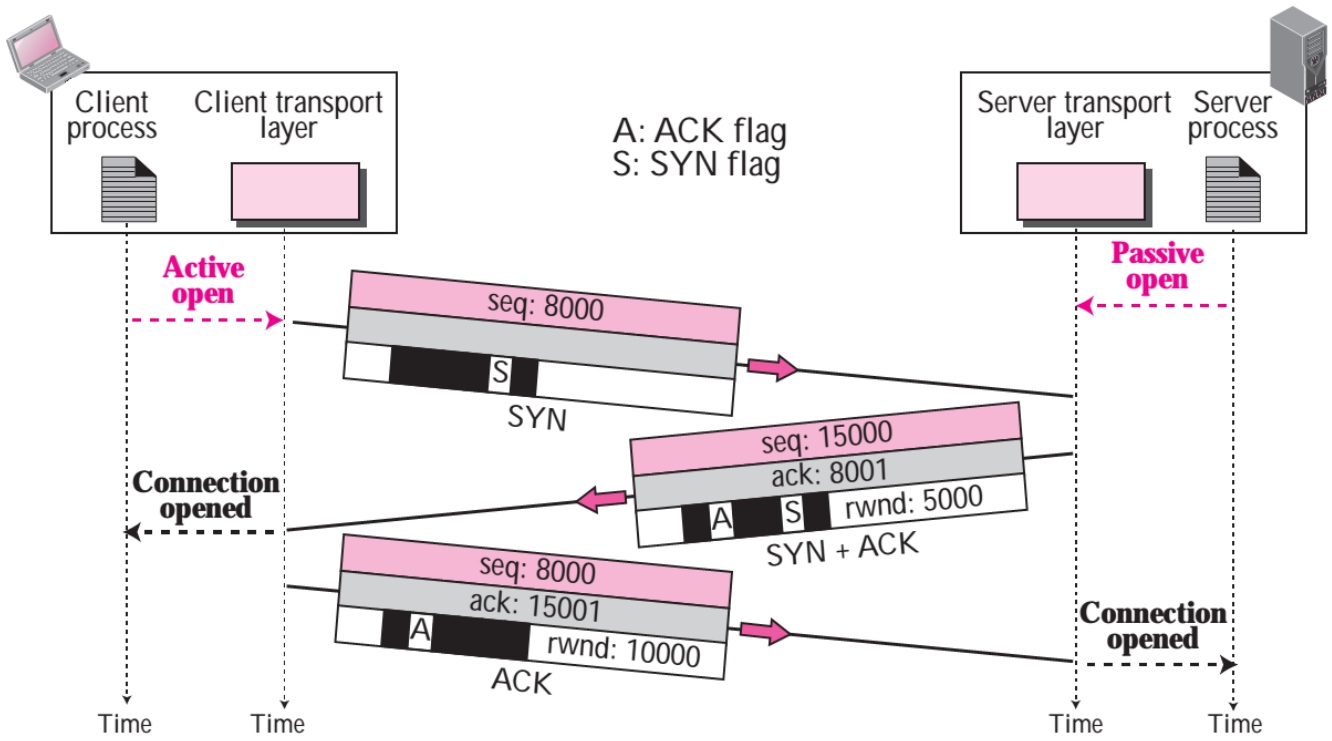
TCP Connection: Establishment

- TCP transmits data in full-duplex mode.
- Each party must initialize communication and get approval from the other party before any data are transferred.

- The connection establishment in TCP is called **three-way handshaking**.

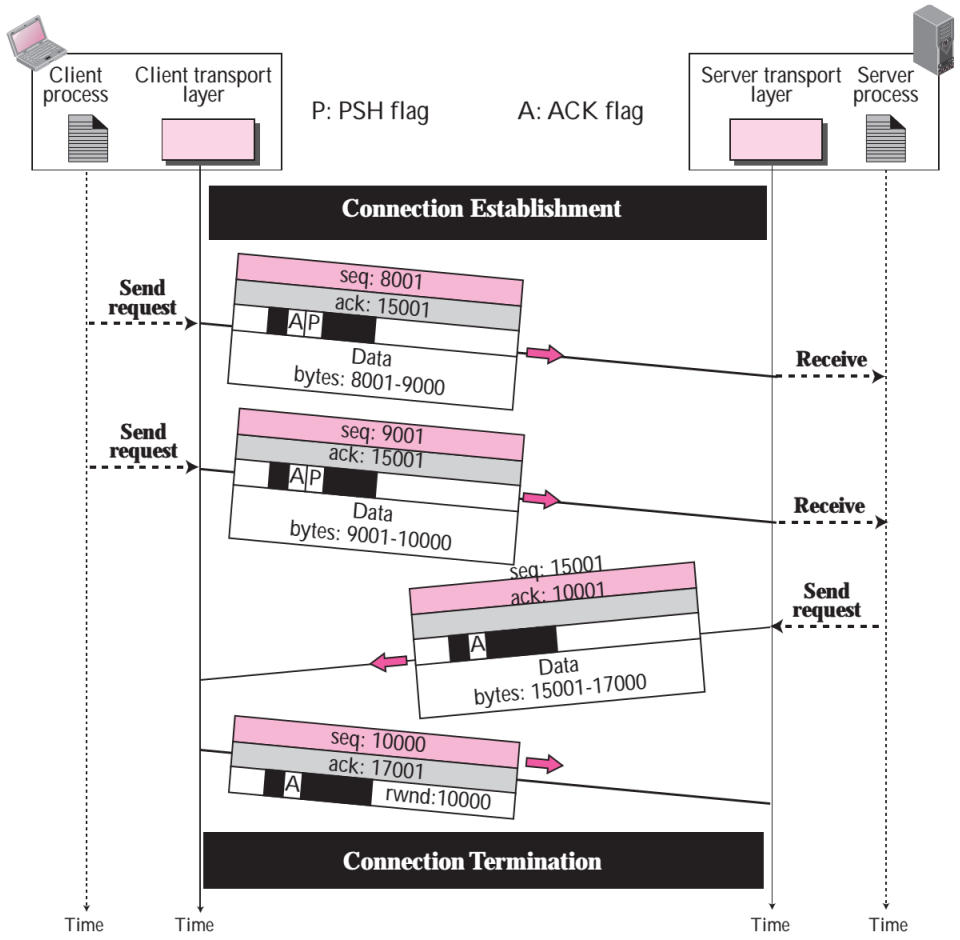
TCP Connection: Establishment

- A SYN segment cannot carry data, but it consumes one sequence number.
- A SYN + ACK segment cannot carry data, but does consume one sequence number.
- An ACK segment, if carrying no data, consumes no sequence number.



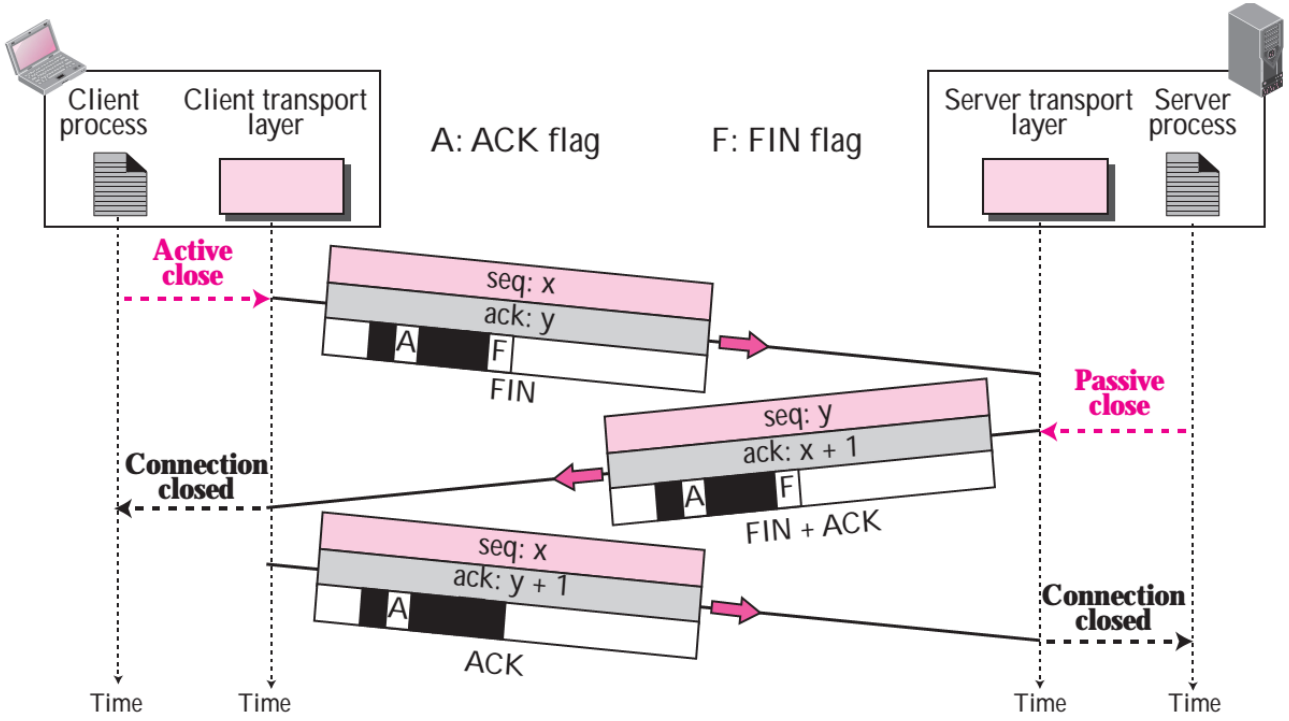
TCP Connection: Data Transfer

- After connection is established, **bidirectional data transfer** can take place.
- The client and server can send data and acknowledgments in both directions.



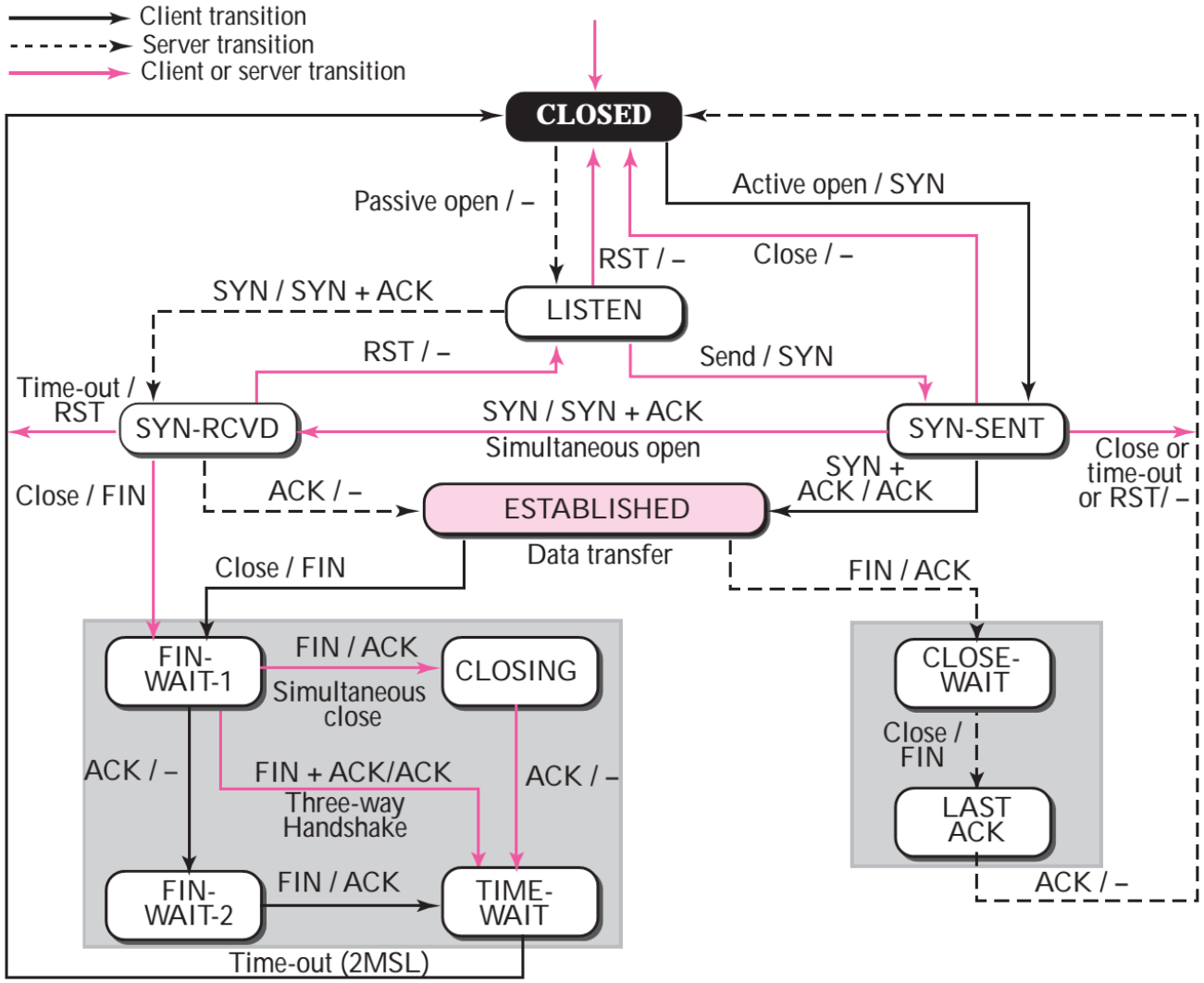
TCP Connection: Termination

- Any of the two parties can close the connection.
  - Although it is usually initiated by the client.
- The FIN segment consumes one sequence number if it does not carry data.
- The FIN + ACK segment consumes one sequence number if it does not carry data.



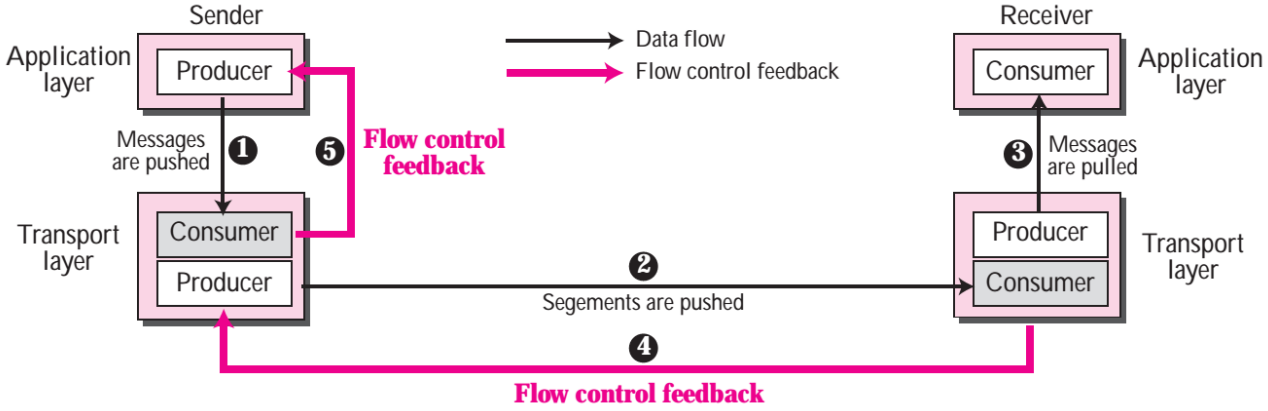
State Transition Diagram

- To keep track of all the different events happening during connection establishment, connection termination, and data transfer, TCP is specified as the finite state machine shown here.



TCP Flow Control

- **Flow Control** balances the rate a producer creates data with the rate a consumer can use the data.
- **The receiving TCP controls the sending TCP**
  - By adjusting windows size on both sender and receiver.
- **The sending TCP controls the sending process.**
  - through simple rejection of data by sending TCP when its window is full



TCP Error Control

- TCP is a reliable transport layer protocol.
- This means that an application program that delivers a stream of data to TCP relies on TCP to deliver the entire stream to the application program on the other end in order, without error, and without any part lost or duplicated.
- **Error control in TCP is achieved through the use of three tools:**
  1. Checksum.
  2. Acknowledgment.
  3. Time-out.

TCP Error Control

1. Checksum:

- Each segment includes a checksum field, which is used to check for a corrupted segment.
- If a segment is corrupted and deleted by an invalid checksum, the segment is discarded by the destination TCP and is considered as lost.
- TCP uses a 16-bit checksum that is mandatory in every segment.

TCP Error Control

2. Acknowledgment:

- TCP uses acknowledgments to confirm the receipt of data segments.
- Control segments that carry no data, but consume a sequence number, are also acknowledged.
- ACK segments do not consume sequence numbers and are not acknowledged.

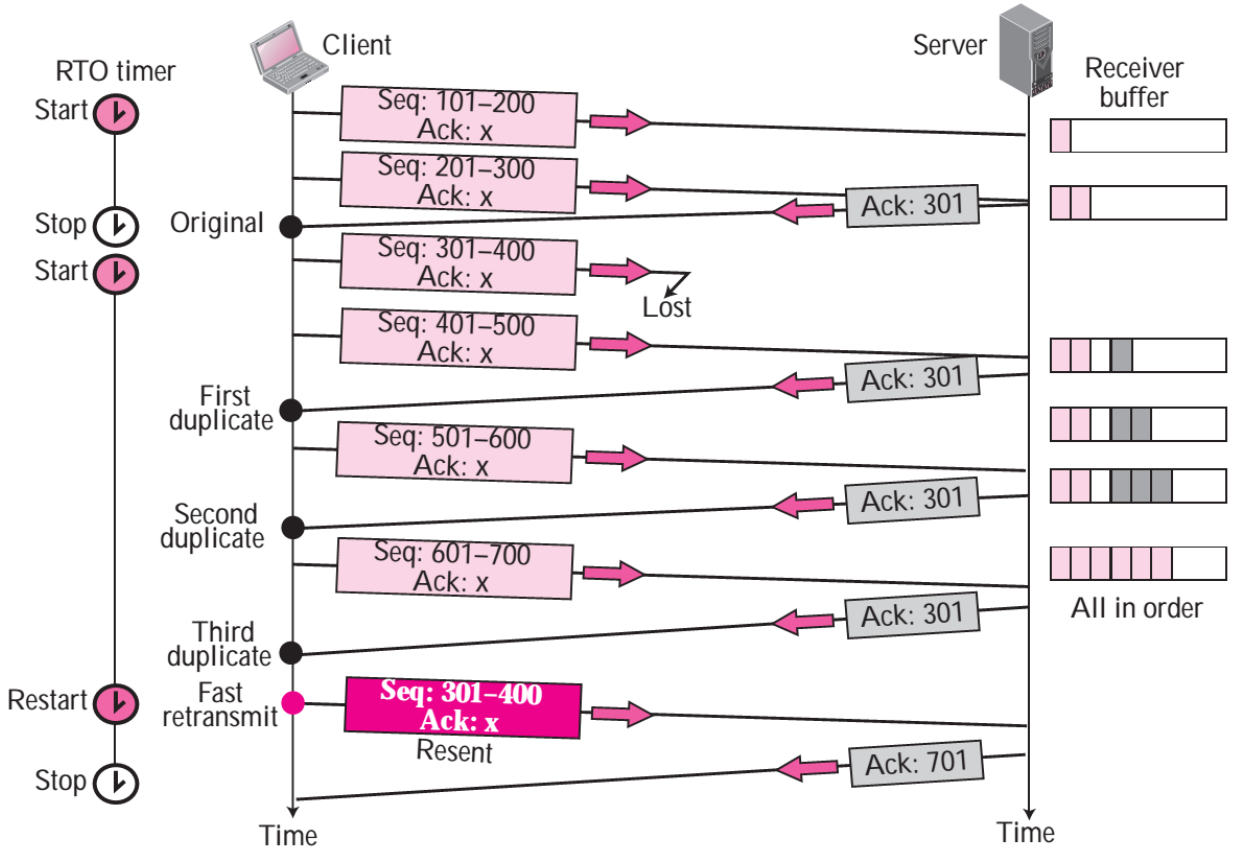
TCP Error Control

3. Retransmission:

- The heart of the error control mechanism is the retransmission of segments.
- When a segment is sent, it is stored in a queue until it is acknowledged.
- The **segment is retransmitted** when any of the following happens:
  1. When the retransmission timer expires.
  2. when the sender receives three duplicate ACKs for the first segment in the queue (fast retransmission).

TCP Error Control

- **Fast Retransmission:**



Out-of-Order Segments

- TCP implementations today do not discard out-of-order segments.
  - They store them temporarily and flag them as out-of-order segments until the missing segments arrive.
- Note, however, that out-of-order segments are never delivered to the process.
- **TCP guarantees that data are delivered to the process in order.**

TCP Congestion Control

- Congestion control in TCP is based on both open loop and closed-loop mechanisms.
- TCP uses a congestion window and a congestion policy that avoid congestion and detect and alleviate congestion after it has occurred.

TCP Congestion Control

- Receiver Window (rwnd): the amount of data that the destination side can receive.
- Congestion Window (cwnd): the amount of data the TCP can send into the network before receiving an ACK
- The sender has two pieces of information. The actual size of the window is the minimum of these two.

Actual window size for the sender = minimum (rwnd, cwnd)

TCP Congestion Control

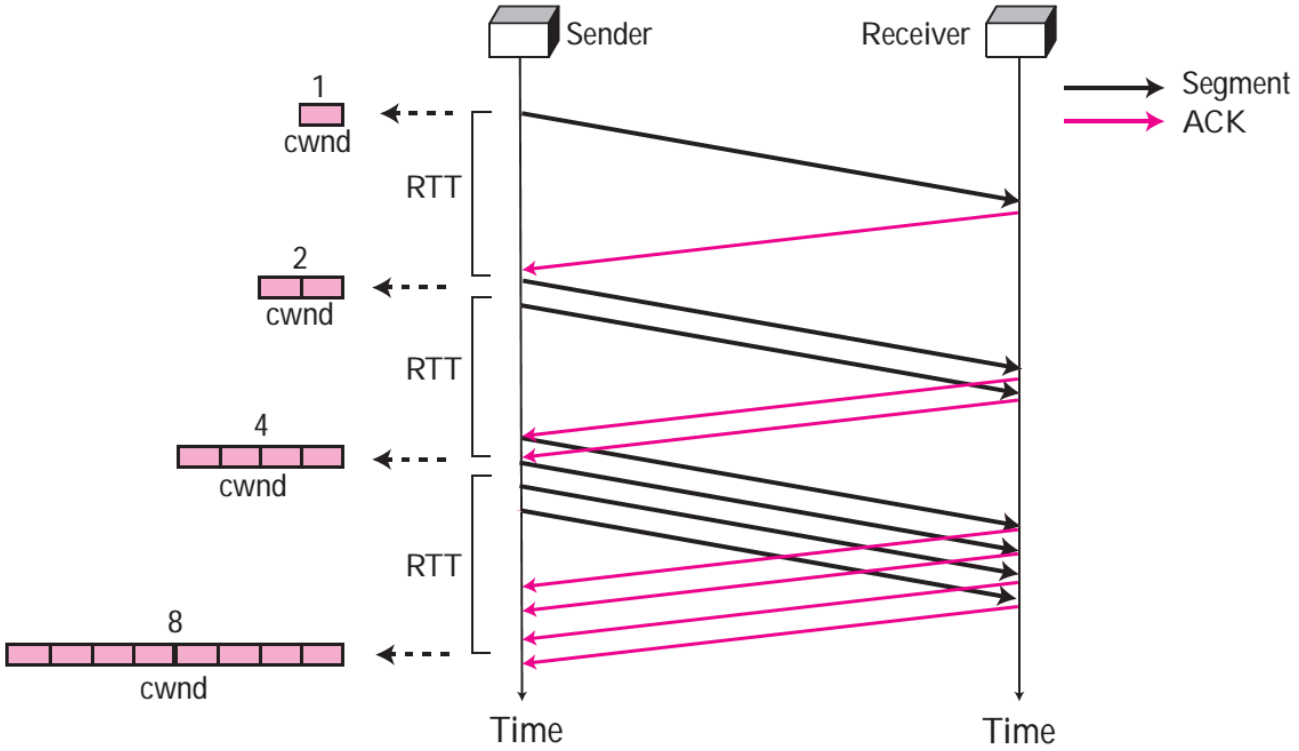
Congestion Policy:

- TCP's general policy for handling congestion is based on three phases:
  1. Slow start.
  2. Congestion avoidance.
  3. Congestion detection.
- In the slow start phase, the sender starts with a slow rate of transmission, but increases the rate rapidly to reach a threshold.
- When the threshold is reached, the rate of increase is reduced.
- Finally if ever congestion is detected, the sender goes back to the slow start or congestion avoidance phase, based on how the congestion is detected.

TCP Congestion Control

- Slow Start: Exponential Increase

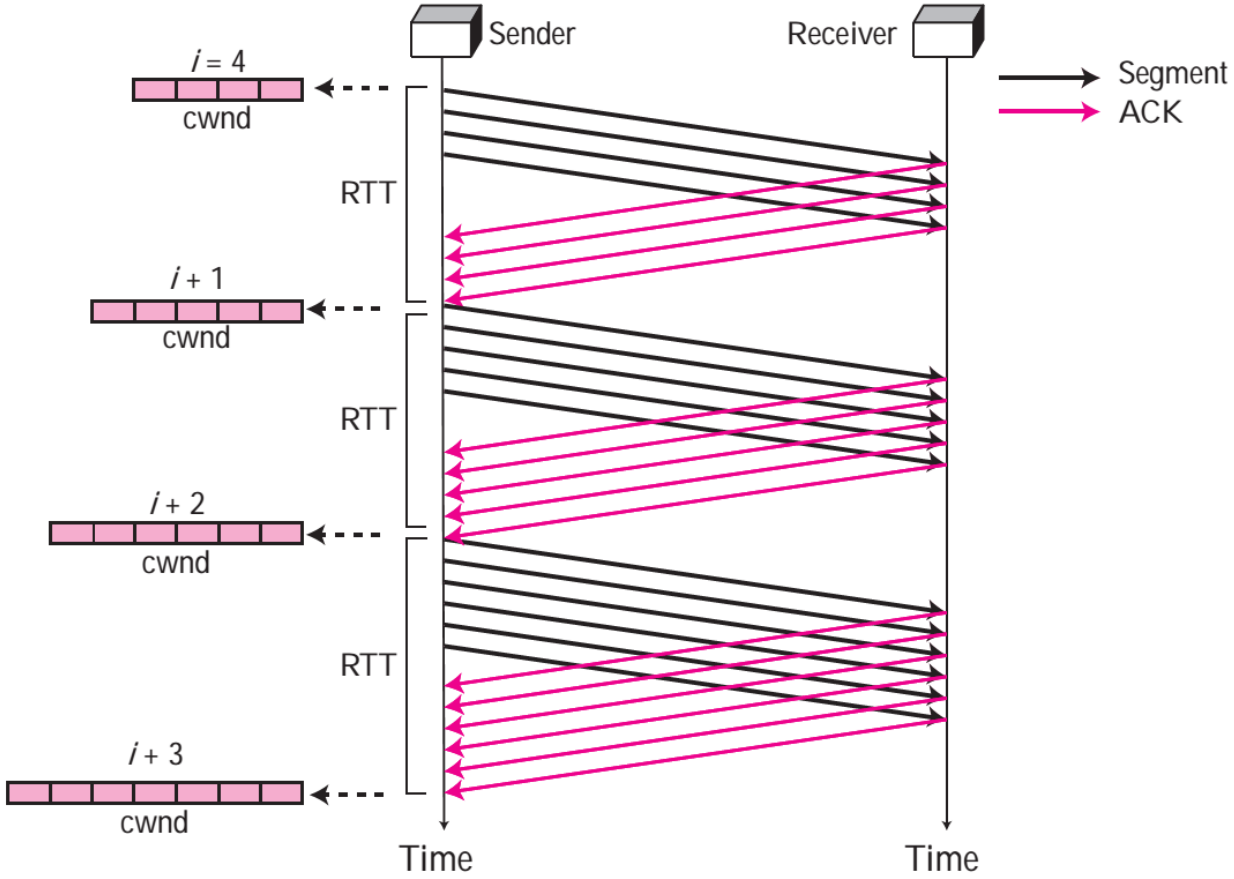
- In the slow start algorithm, the size of the congestion window increases exponentially until it reaches a threshold.



TCP Congestion Control

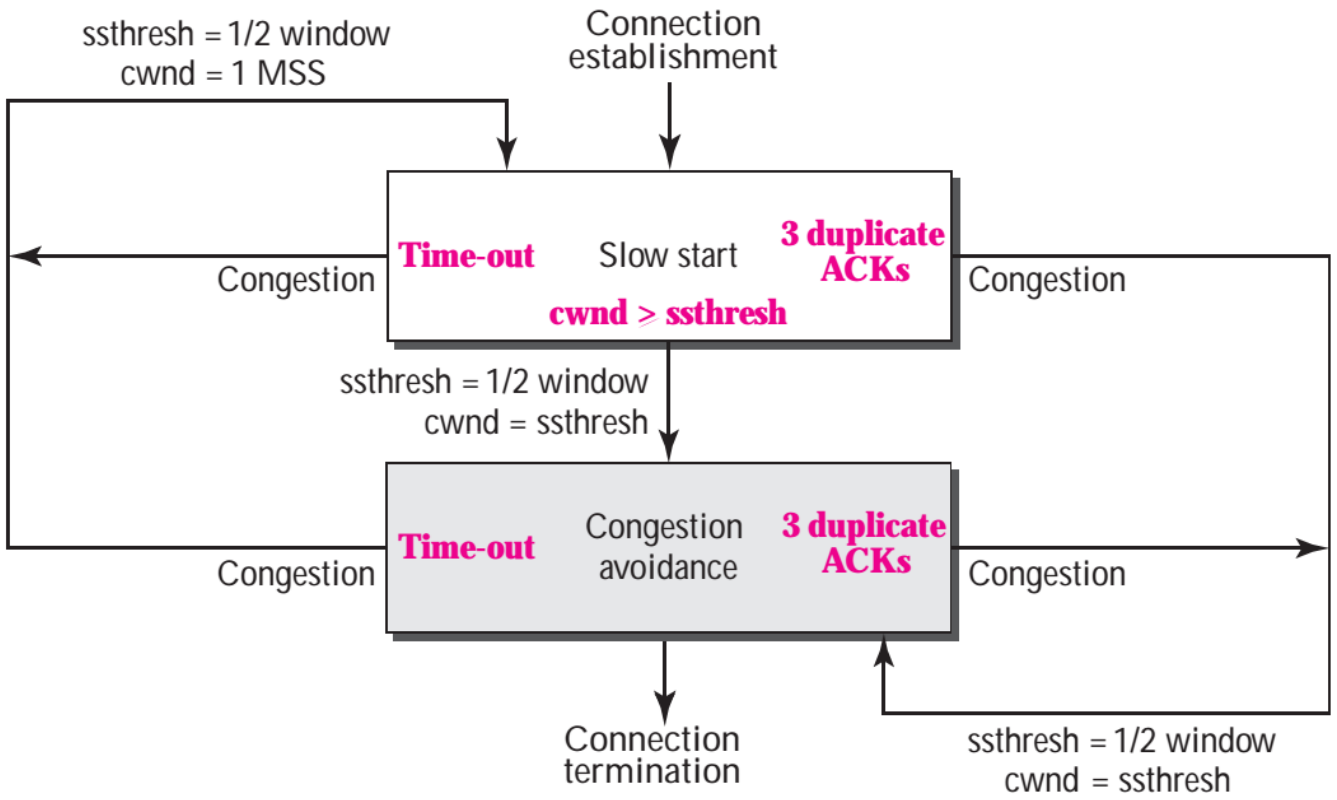
- Congestion Avoidance: Additive Increase

- In the congestion avoidance algorithm the size of the congestion window increases additively until congestion is detected.

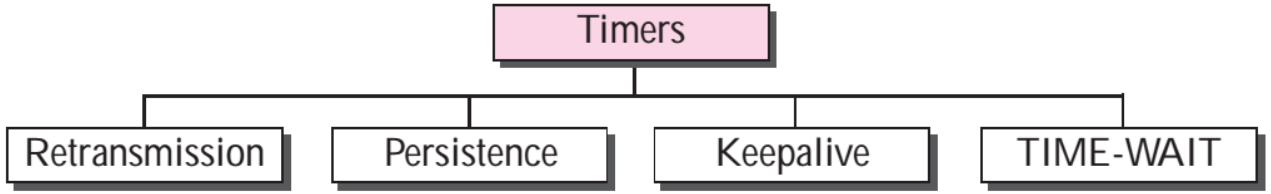


TCP Congestion Control

- TCP congestion policy summary:



TCP Timers



To perform its operation smoothly, most TCP implementations use at least four timers:

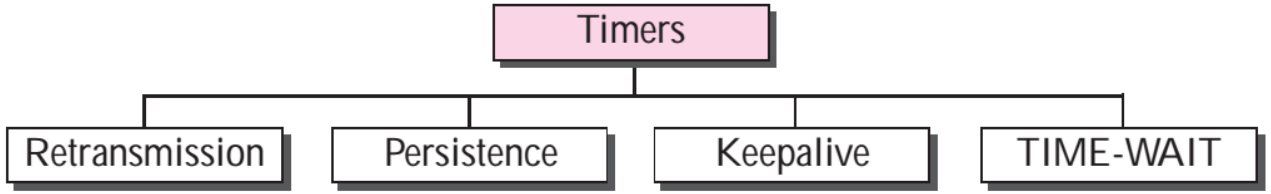
1. Retransmission Timer:

- Waiting time the sender need to wait for an acknowledgment of a segment before retransmission.

2. Persistence Timer:

- If the receiving TCP announces a window size of zero, the sending TCP stops transmitting segments until the receiving TCP sends an ACK segment announcing a nonzero window size.
- This ACK segment can be lost. Both TCPs might continue to wait for each other forever (a deadlock).
- The sending TCP starts a persistence timer when it receives a an acknowledgment with a window size of zero.

TCP Timers



3. Keepalive Timer:

- To prevent a long idle connection between two TCPs.
- For example:
  - If the server does not hear from the client after 2 hours, it sends a probe segment. If there is no response after 10 probes, it assumes that the client is down and terminates the connection.

4. TIME-WAIT Timer:

- Is used during connection termination